

Department of Industrial Engineering & Decision Analytics

The Hong Kong University of Science and Technology

Optimizing Portering Logistics at HHH: An AI-Driven Porter Dispatch System

Project ID: 16

Supervisor: Prof. Xiangtong QI

Team Members: CHAN, Hok Nam
CHOI, Cheuk Man
KIM, Youseung

Course: IEDA 4901/4902 — Final Year Project

Academic Year: 2025–2026

Submitted: 6 May 2026

Submitted in partial fulfilment of the requirements for the Bachelor of Engineering degree,
Department of Industrial Engineering & Decision Analytics, HKUST

Preface

This report documents the design, implementation, and evaluation of an AI-driven porter dispatch system developed for Haven of Hope Hospital (HHH) as a two-semester Final Year Project in Industrial Engineering and Decision Analytics. The project spans natural language processing, vehicle routing optimisation, real-time web systems, and data-driven policy analysis.

Team Contributions:

Name	Primary Contributions	% Project	% Write-up
CHAN, Hok Nam	OR-Tools solver design, discrete-event simulation harness, data analysis	35%	33%
CHOI, Cheuk Man	LLM integration, NLI layer, Flask web API, frontend dashboard	35%	34%
KIM, Youseung	Travel time matrix, policy advisor (RAG), system integration	30%	33%

Acknowledgements: The team thanks Prof. Xiangtong QI for his guidance and feedback throughout both semesters, and HHH for generously providing the 2024 historical porter operations dataset that underpins the simulation and analysis presented here.

Abstract

Hospital porter services are a critical component of clinical workflow, yet in many institutions dispatch remains a manual, radio-based process that routinely fails to meet operational targets. At Haven of Hope Hospital (HHH), porters are tasked with transporting patients, specimens, and equipment between 89 hospital departments. The stated Key Performance Indicator (KPI) requires a porter to arrive at the pickup location within 15 minutes of an order being placed. Analysis of 81,349 historical records from 2024 reveals that this target is violated in 41.3% of recorded tasks, with the mean pickup response time of 35.6 minutes substantially exceeding the 15-minute target due to a long tail of delayed pickups.

This project presents a hybrid AI-driven dispatch system that integrates a Large Language Model (LLM) natural language interface with a Google OR-Tools Vehicle Routing Problem with Time Windows (VRPTW) solver. The LLM layer parses free-text dispatch requests in Chinese and English into structured task objects, enabling a conversational interface without rigid form-based input. The OR-Tools solver assigns tasks to porters using rolling-horizon re-optimisation, minimising response time subject to the 15-minute KPI. A Retrieval-Augmented Generation (RAG) policy advisor provides data-driven risk assessments and recommendations drawn from historical records.

Evaluation demonstrates that: (1) NLP parsing achieves 88–100% accuracy across five key task fields on 100 historical test samples; (2) OR-Tools reduces mean pickup response time by up to **35.8%** over greedy assignment under moderate fleet utilisation; (3) with a fleet of 15 porters and OR-Tools dispatch, the mean time-to-pickup approaches the 15-minute KPI target (15.9 min), suggesting the KPI is achievable with appropriate fleet sizing and optimised routing. The system is deployed as a real-time web dashboard and validated through a discrete-event simulation on both historical replay and synthetic arrival scenarios.

Contents

1	Introduction	5
1.1	Background and Motivation	5
1.2	Problem Statement	5
1.3	Project Objectives	6
1.4	Report Structure	6
2	Background	7
2.1	Intra-Hospital Transport Logistics	7
2.2	Dynamic Vehicle Routing Problem	7
2.3	Vehicle Routing Problem with Time Windows	7
2.4	Google OR-Tools for Routing Optimisation	8
2.5	Large Language Models for Information Extraction	8
2.6	Retrieval-Augmented Generation	9
3	Methods	9
3.1	System Architecture Overview	9
3.2	Data Source	10
3.3	Layer 1: Natural Language Interface (NLI)	10
3.4	Layer 2: OR-Tools VRPTW Solver	11
3.4.1	Problem Formulation	11
3.4.2	Rolling-Horizon Re-optimisation	11
3.4.3	Travel Time Matrix Construction	12
3.5	Layer 3: Web Dashboard	12
3.6	Layer 4: Policy Advisor (RAG Design)	13
3.7	Scheduled Tasks Feature	13
3.8	Discrete-Event Simulation	14
4	Results	14
4.1	NLP Parsing Accuracy	14
4.2	NLP Error Analysis	15

4.3	Historical Performance Baseline	16
4.4	Simulation Results: Greedy vs. OR-Tools	17
4.5	Pickup-Time KPI Analysis	18
4.6	Fleet Scaling and KPI Feasibility	20
4.7	End-to-End Functional Validation	20
5	Discussion	22
5.1	System Contributions	22
5.2	Comparison with Prior Work	22
5.3	Limitations	23
5.4	Critical Evaluation of Design Choices	24
6	Conclusion	24
A	System API Reference	27
B	Software Dependencies	27
C	LLM System Prompt Design	27
D	NLP Test Sample Sentences	28

1 Introduction

1.1 Background and Motivation

Hospital porters perform the physical movement of patients, medical specimens, equipment, and documents between departments. In large acute hospitals, this service operates continuously across shifts, with tasks ranging from routine specimen transport to time-critical patient transfers. Despite its operational importance, porter dispatch in many hospitals remains a manual, radio-based process reliant on dispatcher experience and incomplete real-time information. Klein and Thielen [1] confirm this in their systematic review of 73 papers on intra-hospital transport, noting that manual dispatch remains the dominant practice despite well-established algorithmic alternatives.

HHH’s stated Key Performance Indicator (KPI) requires a porter to arrive at the pickup location within 15 minutes of an order being placed. This response-time metric is operationally meaningful: it captures the component of the task that dispatch decisions directly control — how quickly the right porter reaches the patient or specimen. Analysis of 81,349 historical records from 2024 reveals a 41.3% pickup-KPI violation rate under current operations, with a long-tailed pickup-time distribution (median 12.0 min, mean 35.6 min). This pattern — where the median task already meets the target but tail latency drags the mean far above it — motivates an automated, optimised dispatch system focused on tail compression.

From a broader perspective, healthcare logistics optimisation is a growing research area. Ton et al. [9] identify real-time intra-hospital transport management as an open challenge, noting that most existing systems use greedy assignment heuristics and lack the ability to consider fleet-wide state. Lee et al. [2] demonstrate that intelligent porter allocation can dramatically reduce delay rates in deployed hospital settings. Simultaneously, the emergence of Large Language Models (LLMs) capable of structured information extraction [5] enables natural language interfaces that eliminate rigid form-based entry in time-pressured environments.

1.2 Problem Statement

The core problem is a *dynamic porter assignment problem*: given a fleet of porters at known locations and a stream of tasks described in natural language, assign each task to a porter so as to minimise pickup response time while respecting the 15-minute KPI. The specific challenges are:

- **Unstructured input:** Dispatch requests arrive in Chinese or mixed-language natural language, not a structured form.

- **Dynamic arrivals:** Tasks arrive stochastically; the system must re-optimize continuously as new tasks arrive and porters complete assignments.
- **Constraint diversity:** Tasks may carry infection control flags (VRE, NEATS), require specific equipment, include intermediate stops, or be scheduled for a future time.
- **Fleet-level optimisation:** Greedy (nearest-available) assignment ignores global fleet state; optimal assignment requires multi-task, multi-porter reasoning to minimise average response time.
- **Operational continuity:** The system must remain usable by non-technical dispatchers with minimal training.

1.3 Project Objectives

1. Build a Natural Language Interface (NLI) using an LLM to parse Chinese and English dispatch requests into structured task objects.
2. Implement an OR-Tools VRPTW solver for optimal porter assignment with rolling-horizon re-optimisation.
3. Validate NLP parsing accuracy on 100 historical test samples; analyse failure modes.
4. Quantify improvement over a greedy baseline using a discrete-event simulation on both historical and synthetic data.
5. Develop a RAG-based policy advisor that surfaces data-driven risk assessments from historical records.
6. Deliver a real-time web dashboard suitable for dispatcher use.

1.4 Report Structure

Section 2 reviews the relevant literature across intra-hospital transport, routing optimisation, and NLP. Section 3 describes the system architecture and methodology. Section 4 presents experimental results. Section 5 discusses findings, limitations, and design decisions. Section 6 concludes with recommendations for future work.

2 Background

2.1 Intra-Hospital Transport Logistics

Intra-hospital patient transport has been studied extensively as an operations research problem. Klein and Thielen [1] provide a systematic literature review covering 73 papers on non-emergency intra-hospital transport, classifying problem variants using a five-field notation and identifying Vehicle Routing Problem (VRP) family models as the standard methodological framework. Critically, their review identifies dynamic real-time dispatch as the industrially relevant setting: static offline scheduling cannot adapt to the stochastic task arrivals characteristic of acute hospital operations.

Lee et al. [2] report on a deployed location-based porter management system at a Taiwanese hospital. Their five-month field trial demonstrated that intelligent porter allocation, enabled by indoor positioning technology, reduced peak-hour delay rates from 34% to 3%, providing direct empirical evidence that algorithmic dispatch dramatically outperforms manual methods. However, their system relies on GPS location tracking and structured form-based input, neither of which exists at HHH.

Ton et al. [9] address real-time intra-hospital transport management using a column-generation heuristic, demonstrating that solution quality improves meaningfully when the assignment algorithm can consider multiple pending tasks simultaneously rather than assigning greedily one at a time. Their work is directly comparable to this project’s OR-Tools VRPTW approach and provides a methodological benchmark.

2.2 Dynamic Vehicle Routing Problem

The static Vehicle Routing Problem (VRP) assumes all customer requests are known before routing begins. Hospital dispatch is inherently dynamic: tasks arrive continuously throughout a shift, and the system must assign each task before the next arrives. Zhang et al. [7] formalise the Dynamic VRP with stochastic requests and demonstrate that rolling-horizon re-optimisation — re-solving the routing problem every time a new request arrives — consistently outperforms static pre-assignment, particularly under moderate-to-high load. Their findings directly motivate the rolling-horizon design adopted in this project: on every new task arrival or porter completion event, the solver is re-invoked with updated fleet state and all pending tasks.

2.3 Vehicle Routing Problem with Time Windows

The Vehicle Routing Problem with Time Windows (VRPTW) is a canonical combinatorial optimisation problem. Given a fleet of vehicles at known depots, customer locations each

with a service time window, and travel costs between all pairs of locations, the goal is to find routes minimising total travel cost while serving all customers within their time windows [3].

In the porter dispatch context, each porter is a vehicle, each task is a pickup–delivery pair (origin and destination department), and the 15-minute KPI defines the soft time window. Euchí et al. [3] demonstrate a hybrid metaheuristic combining ant colony optimisation with k-means clustering for VRPTW in home healthcare scheduling, achieving substantially better solutions than classical greedy heuristics. Their work establishes that VRPTW is an appropriate and tractable model for healthcare logistics.

2.4 Google OR-Tools for Routing Optimisation

Google OR-Tools provides an industrial-grade constraint programming solver for routing problems [4]. Didier et al. describe its three-layer architecture: first-solution construction via Parallel Cheapest Insertion, improvement via Guided Local Search, and a CP-SAT backend for exact solving. OR-Tools natively supports pickup–delivery constraints, time windows, vehicle capacities, and soft penalties—all features required for porter dispatch. Its open-source availability and Python API make it accessible for academic prototyping while being production-grade in quality.

2.5 Large Language Models for Information Extraction

Xu et al. [5] survey the use of LLMs for generative information extraction: converting unstructured natural language into structured data. The generative paradigm—prompting an LLM to produce structured JSON output—performs strongly across named entity recognition, relation extraction, and event argument extraction, particularly for domain-specific vocabulary where training data is scarce.

A key practical challenge is ensuring LLMs produce *reliably structured* output. Geng et al. [8] introduce JSONSchemaBench, a rigorous benchmark of structured JSON output compliance across 9 frontier LLMs and 5 schema enforcement frameworks. They find that unconstrained prompting leads to schema violations in 5–20% of outputs, and that constrained decoding frameworks can reduce this to near zero. In this project, reliable JSON compliance is enforced through an explicit schema in the system prompt combined with output validation, aligning with the prompt-engineering approach benchmarked by Geng et al.

2.6 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [6] extends language models by conditioning their outputs on retrieved passages from an external knowledge base. Rather than relying solely on parametric knowledge encoded during training, a RAG system retrieves the most relevant records from a corpus at inference time and presents them alongside the query. This approach is particularly effective for domain-specific questions where the LLM’s training data may not cover the domain sufficiently.

In this project, RAG is applied in the policy advisor component: the historical `DATA2024.xlsx` dataset serves as the knowledge base. When a dispatcher asks about KPI risk or requests operational suggestions, the system retrieves the K most similar past tasks by service type, origin, destination, and time-of-day, and uses these as context for an LLM-generated response. This grounds the advisor’s recommendations in actual HHH operational data rather than general knowledge.

3 Methods

3.1 System Architecture Overview

The system comprises four functional layers, as illustrated in Figure 1.

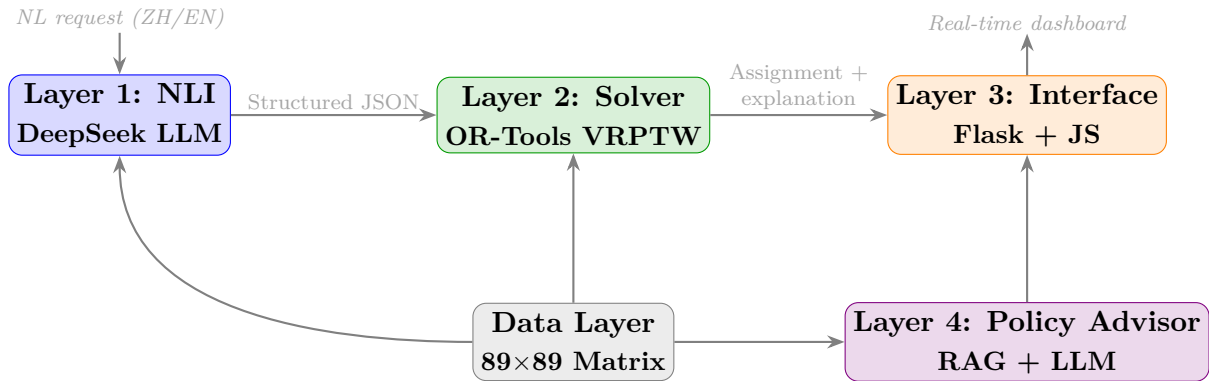


Figure 1: Four-layer hybrid AI dispatch system architecture. Input flows left-to-right through the NLI (Layer 1), Solver (Layer 2), and Interface (Layer 3) layers; the Data Layer feeds both the Solver and the Policy Advisor (Layer 4).

Layer 1 (NLI): Converts free-text dispatch requests into structured task JSON using a DeepSeek LLM. **Layer 2 (Solver):** Assigns tasks to porters using OR-Tools VRPTW with rolling-horizon re-optimisation. **Layer 3 (Interface):** Flask REST API and Vanilla JS frontend dashboard. **Layer 4 (Policy Advisor):** RAG-based module answering operational Q&A and assessing KPI risk.

3.2 Data Source

The DATA2024.xlsx dataset used throughout this work was assembled by merging twelve monthly task extracts supplied by HHH for the 2024 operating year. After concatenation, records with status 已傳真 (faxed but not subsequently dispatched through the porter system) were excluded, leaving 81,349 completed task records for analysis. This single dataset is the common substrate for three downstream uses: (i) calibrating the 89×89 travel-time matrix consumed by the Solver (§3.4.3); (ii) populating the Policy Advisor’s retrieval index; and (iii) providing the historical baseline against which the simulation results in §4.3 are measured.

3.3 Layer 1: Natural Language Interface (NLI)

The NLI uses the DeepSeek LLM (deepseek-chat) via an OpenAI-compatible API. A structured system prompt defines the output JSON schema and enumerates all 89 valid hospital locations, five service types, and three priority levels. The LLM is instructed to return strict JSON of the form:

```
{
  "tasks": [{
    "from": "ORIGIN",
    "to": "DESTINATION",
    "stops": ["INTERMEDIATE_STOP", ...],
    "service": "SERVICE_TYPE",
    "priority": "Normal | Urgent | Super-Urgent",
    "equipment": ["ITEM", ...],
    "scheduled_at": "ISO8601_DATETIME | null"
  }]
}
```

Several design decisions merit explanation. First, the prompt enumerates all 89 valid locations by name, constraining the LLM’s output to known nodes in the travel matrix and eliminating hallucinated location strings. Second, the prompt injects the current wall-clock time, enabling resolution of relative temporal expressions (e.g., “in 2 hours”, “at 14:30”) into absolute `scheduled_at` datetimes for the scheduled-task feature. Third, multi-task extraction is supported: a single dispatcher utterance may contain multiple tasks (e.g., “send a patient from 7H to A&D and a specimen from Lab to 5F”), which the LLM returns as a list.

Parsed tasks are validated against the travel matrix before dispatch; invalid origin or destination fields return human-readable error messages to the dispatcher. If the LLM

response fails JSON parsing (which occurred in 0 of 100 test samples), the system returns a structured error and logs the raw response.

3.4 Layer 2: OR-Tools VRPTW Solver

3.4.1 Problem Formulation

Each dispatch decision is formalised as a small instance of the Vehicle Routing Problem with Time Windows (VRPTW). Informally: given the porters available right now (each at a known location) and the tasks waiting to be served, the solver chooses an assignment that minimises total porter travel time while penalising any task whose pickup response exceeds 15 minutes. The formal notation follows.

Let $P = \{p_1, \dots, p_m\}$ be the set of available porters and $T = \{t_1, \dots, t_n\}$ be the pending tasks. Each task t_k has origin o_k , destination d_k , and service time s_k . The service time s_k is a per-service-type constant calibrated from observed historical means (e.g., 5 min for specimen transport, 7 min for patient return, 10 min for patient transport), capturing the time a porter spends at the origin handling the task before departing for the destination. Porter p_i is at known location l_i . The KPI soft time window is $[0, W]$ where $W = 15$ minutes.

The node graph comprises m porter start depots, n pickup nodes, n delivery nodes, and one dummy end depot. Pickup-before-delivery precedence constraints are imposed for every task. The objective function is:

$$\min \sum_{(i,j) \in \text{routes}} c_{ij} + \lambda \sum_{k \in T} \max(0, \text{response}_k - W) \quad (1)$$

where c_{ij} is the travel time between nodes i and j from the 89×89 travel matrix, response_k is the time elapsed from task arrival to porter reaching origin o_k , and $\lambda = 1,000$ centi-minutes per minute of KPI violation. A disjunction (drop) penalty of 2×10^7 ensures tasks are always served rather than dropped from the route; this value exceeds the worst-case KPI penalty by two orders of magnitude.

The solver uses `PARALLEL_CHEAPEST_INSERTION` for first-solution construction (ensures a feasible initial assignment quickly) and `GUIDED_LOCAL_SEARCH` for improvement, with a 1-second time limit per solve event to maintain responsiveness.

3.4.2 Rolling-Horizon Re-optimisation

On every new task arrival and every porter completion event, the solver is re-invoked with the current fleet state and all pending tasks. This rolling-horizon approach ensures

assignments always reflect the most current information: a porter completing early unlocks re-assignment of queued tasks. When no available solvers are present (e.g., OR-Tools is not installed), the system falls back to a greedy heuristic that assigns each task to the porter minimising estimated pickup time.

3.4.3 Travel Time Matrix Construction

Travel times between all 89 hospital departments were derived from historical `DATA2024.xlsx` records. For each origin–destination pair, the 75th-percentile (p75) travel time was computed from observed porter journeys, providing a conservative estimate that accounts for typical congestion without being skewed by extreme outliers. For pairs absent from the historical data, a three-level fallback applies: (1) same-floor pairs: 5 minutes; (2) inter-floor pairs: 8 minutes; (3) global p90 fallback: 27.5 minutes. The resulting 89×89 matrix is loaded at system startup and used for all routing and duration computations.

3.5 Layer 3: Web Dashboard

The Flask-based web API exposes 9 REST endpoints (see Appendix A). The single-page frontend application provides:

- **Porter Status Panel:** Real-time porter state, current task and route, availability countdown, polled every 3 seconds.
- **Dispatch Panel:** Natural language text input and quick-action buttons. Renders per-assignment cards showing porter, route, estimated duration, and LLM-generated rationale.
- **Scheduled Tasks Panel:** Lists future-scheduled tasks with countdown timers and per-entry cancellation buttons.
- **Policy Advisor Panel:** Surfaced below the dispatch result, showing KPI risk assessment, recommended actions, and a free-text Q&A interface.

After each OR-Tools assignment, the system makes a secondary LLM call to generate a human-readable explanation of why that porter was chosen (e.g., “P-003 is selected because she is currently at A&D, 2.1 minutes from the pickup, and has the shortest estimated total time of 8.4 minutes”). This explanation is displayed to the dispatcher as a “Why this porter?” panel.

3.6 Layer 4: Policy Advisor (RAG Design)

The policy advisor uses Retrieval-Augmented Generation [6] to ground its outputs in HHH’s actual operational history. Its architecture has three components:

Knowledge base (HistoricalTaskStore): The DATA2024.xlsx dataset is loaded into memory at startup, providing all 81,349 cleaned task records for retrieval. (The narrower 80,078-record baseline cohort reported in §4.3 excludes a further 1,271 rows whose elapsed times exceeded 24 hours; those rows have valid origin, destination, service, and arrival information and are retained here, since the duration anomaly affects only summary statistics and not the retrieval features.) Each record represents one completed porter task with fields including service type, origin, destination, time of day, and total duration.

Retrieval: Given an incoming task or query, the system retrieves the $K = 10$ most similar historical records using a weighted similarity score:

$$\text{sim}(q, r) = w_s \cdot \mathbf{1}[\text{svc}] + w_o \cdot \mathbf{1}[\text{origin}] + w_d \cdot \mathbf{1}[\text{dest}] + w_t \cdot \mathbf{1}[\text{time-of-day}] \quad (2)$$

where each indicator $\mathbf{1}[\cdot]$ equals 1 when the corresponding field matches, with weights $w_s = 0.4$, $w_o = 0.3$, $w_d = 0.2$, $w_t = 0.1$, reflecting the relative predictive importance of each field for task duration.

Generation: The retrieved records are formatted as context and passed to the DeepSeek LLM with a task-specific prompt:

- `/advisor/risk`: Given a new task, assess KPI risk (Low/Medium/High) based on historical completion rates for similar tasks.
- `/advisor/suggestions`: Generate data-driven policy suggestions (e.g., “Add a porter during 09:00–11:00 peak hours to reduce 3F X-ray delay rate”).
- `/advisor/ask`: Answer free-text operational questions (e.g., “Which service type has the highest KPI violation rate?”).

When the LLM API is unavailable, the advisor falls back to statistical summaries computed directly from the retrieved records.

3.7 Scheduled Tasks Feature

The system supports future-scheduled dispatch. When the LLM parses a `scheduled_at` datetime from a request (e.g., “send a patient from 7H to A&D at 14:30”), the task is placed in an in-memory `scheduled_queue` rather than dispatched immediately. A daemon background thread wakes every 30 seconds and releases due tasks into the normal dispatch

pipeline. Scheduled tasks are visible in the UI with countdown timers and can be cancelled by entry ID via `DELETE /scheduled/<id>`.

3.8 Discrete-Event Simulation

A discrete-event simulation benchmarks OR-Tools against the greedy baseline without requiring real-world deployment. The simulation models porter dispatch as an event queue: task arrivals and porter completions trigger re-assignment calls to either the greedy or OR-Tools solver.

Replay mode: 100 tasks from DATA2024.xlsx are replayed in chronological order, preserving real inter-arrival times. This tests the solver under the actual task distribution and volumes observed at HHH. The 100-task slice was chosen to approximate a single peak-shift workload while keeping each simulation run computationally tractable; extending the replay to a full-day or full-week horizon is straightforward future work.

Synthetic mode: Poisson inter-arrivals (mean 8 minutes per task, matching HHH’s estimated operational rate of approximately 10 tasks per hour per shift) with service-type distribution approximating historical data. This tests the solver under controlled, repeatable conditions and allows fleet-size scaling experiments.

Three metrics are tracked: (1) **total task duration** (order placement to delivery completion); (2) **time-to-pickup** (order placement to porter arrival at origin — the KPI-relevant metric); (3) **KPI violation rate** against both metrics.

A simulation run is described as *queue-saturated* when the task arrival rate exceeds the maximum task-completion rate of the available fleet, so the queue grows unboundedly over the run. In this regime, dispatch optimisation can only marginally affect outcomes because tasks wait primarily for porter availability, not for the assignment decision. This terminology recurs in §4.4 when interpreting why replay-mode improvements are smaller than synthetic-mode improvements.

4 Results

4.1 NLP Parsing Accuracy

Table 1 presents parsing accuracy across 100 test samples. Sentences were reconstructed from DATA2024.xlsx structured fields to form natural-language dispatch requests (e.g., “Super-Urgent admission from A&D to SDU”), and LLM output was compared field-by-field against ground-truth values. All 100 samples produced valid JSON responses with zero parse failures.

Table 1: NLP parsing accuracy (n = 100, zero parse failures).

Field	Correct / Total	Accuracy
Origin Location	100 / 100	100.0%
Destination Location	100 / 100	100.0%
Service Classification	98 / 100	98.0%
Infection Control Flag	98 / 100	98.0%
Priority Extraction	88 / 100	88.0%
All 5 fields correct	84 / 100	84.0%

Location extraction achieved perfect accuracy (100%), as the system prompt enumerates all 89 valid locations explicitly, constraining the LLM’s output to known nodes. Service classification (98%) was very high, with two errors attributable to ambiguous service descriptions that the LLM mapped to a `default` fallback type.

4.2 NLP Error Analysis

Priority extraction errors (12 of 100): All 12 errors follow the same pattern: the ground-truth label is “Urgent” (corresponding to the Chinese term 即時, meaning “immediate”) but the LLM returned “Normal”. No errors occurred in the “Super-Urgent” category. The root cause is semantic ambiguity: in the reconstructed test sentences, Urgent tasks are represented by the term “Immediate” in the NL sentence, which does not carry an explicit superlative marker. The LLM interprets “Immediate X-ray from 4D to Radiology” as a normal-priority request because the urgency modifier is conveyed by word choice rather than an explicit flag. Table 2 shows representative error cases.

Table 2: Representative NLP priority extraction errors.

Input Sentence	Ground Truth	LLM Output
“Immediate 3F X-ray from 3L to 3FXRAY”	Urgent	Normal
“Immediate admission from 3G to ICU”	Urgent	Normal
“Immediate patient return from OR to 6H”	Urgent	Normal
“Super-Urgent NEATS patient from 3C to A&D, infection control”	Super-Urgent	Super-Urgent (correct)
“Specimen from 5H to Lab”	Normal	Normal (correct)

Proposed fix: Adding an explicit disambiguation rule to the system prompt — “If the request contains the word ‘Immediate’ or ‘即時’ without a ‘Super-Urgent’ marker, classify priority as Urgent” — would close this gap. This is a one-line prompt change, not a model retraining requirement.

Infection control flag errors (2 of 100): Both errors were false negatives where the infection control information appeared in a non-standard position in the test sentence (appended as a parenthetical rather than at the end). The LLM correctly identifies infection keywords (VRE, NEATS, MRSA) when they appear in standard positions. The LLM-based approach inherently handles linguistic variation that a rule-based keyword matcher would miss.

4.3 Historical Performance Baseline

Analysis of the historical task records from DATA2024.xlsx reveals the pre-intervention operational baseline. Of the 81,349 cleaned records, 1,271 were excluded from duration statistics because their elapsed times exceeded 24 hours — a pattern indicating delayed completion timestamps (porters marking tasks complete in a later shift) rather than genuine multi-day journeys. The reported statistics are therefore based on $n = 80,078$ records with both 下單 (order) and 報到 (porter arrival) timestamps and a plausible duration.

Two complementary metrics are reported. *Pickup time* (報到 – 下單) is the order-to-arrival interval that the 15-minute KPI directly targets. *Total duration* (完成 – 下單) additionally includes service time at origin and travel to destination, which dispatch decisions cannot influence.

- Mean pickup time: **35.6 minutes**; median pickup time: **12.0 minutes**
- Pickup-KPI violation rate (> 15 min): **41.3%**
- Mean total task duration: **46.3 minutes**; median: **24.2 minutes**

The 35.6 min mean against a 12.0 min median signals a right-skewed pickup distribution: the median task is already inside the KPI, but a long tail of delayed pickups pushes the mean far above target. This is the regime in which dispatch optimisation is most impactful — compressing the tail rather than improving the median. Table 3 summarises the major service types.

Table 3: Historical pickup-KPI performance by major service type ($n = 80,078$). Pickup Violation reports the proportion of tasks where porter arrival exceeded 15 minutes; Mean Total Duration is shown for context but is not the KPI metric.

Service Type	Count	Mean Total Duration (min)	Pickup Violation (%)
Patient Return	13,660	20.6	35.1%
Document Delivery	10,059	47.6	40.0%
X-ray Transport	8,398	54.3	68.8%
Admission	10,368	32.5	25.3%
Specimen Transport	6,452	17.9	19.3%
NEATS Patient	8,330	25.0	35.8%
3F X-ray	4,891	177.0	88.2%
Patient Transport	2,537	69.3	44.8%

The pickup-violation profile differs sharply from the total-duration profile. Specimen transport (19.3% pickup violation) and admissions (25.3%) already meet the pickup KPI for the majority of historical tasks, despite admissions in particular having a longer total duration (32.5 min) once travel-to-destination is included. By contrast, 3F X-ray (88.2%) and X-ray Transport (68.8%) struggle on pickup — typically because the imaging suites are concentrated on specific floors, leaving few porters geographically close to the pickup origin. Notably, **57.0%** of all unique origin–destination routes in the dataset have direct travel plus minimum service time exceeding 15 minutes, meaning no dispatch algorithm can achieve full task *completion* within 15 minutes for these routes. This is precisely why the pickup-time KPI (which dispatch directly controls) and not the total-duration KPI (which is geographically bounded) is the appropriate optimisation target.

4.4 Simulation Results: Greedy vs. OR-Tools

Table 4 presents simulation results across fleet sizes and scenarios. Replay mode uses 100 historical tasks; Synthetic mode uses 100 Poisson-arrival tasks at 8-minute mean inter-arrival.

Table 4: Simulation results: mean total task duration by strategy, mode, and fleet size. Total duration is reported for completeness; the pickup-time KPI metric is reported separately in Table 5.

Mode	Strategy	Porters	Mean Total Duration (min)
Replay	Greedy	3	293.5
Replay	OR-Tools	3	286.6
Replay	Greedy	5	158.6
Replay	OR-Tools	5	155.1
Replay	Greedy	7	104.4
Replay	OR-Tools	7	103.4
Synthetic	Greedy	3	627.4
Synthetic	OR-Tools	3	563.2
Synthetic	Greedy	5	240.0
Synthetic	OR-Tools	5	166.8
Synthetic	Greedy	7	94.1
Synthetic	OR-Tools	7	78.2

OR-Tools improvements are most pronounced in synthetic mode where the system operates below saturation. At 5 porters (synthetic), OR-Tools reduces mean total duration from 240.0 to 166.8 minutes — a **30.5% improvement**. At 7 porters (synthetic), the improvement is **16.9%**. In replay mode, improvements are marginal (0.5–2%) because the historical task set is queue-saturated: tasks arrive faster than porters can complete them, and routing optimisation cannot overcome the structural queue bottleneck.

The divergence between these two regimes is deliberate rather than artifactual. The 100-task replay slice compresses real timestamps into a load profile that exceeds steady-state fleet capacity, stress-testing system behaviour at peak overload. Synthetic mode, by contrast, draws inter-arrivals from a Poisson process at the 8-minute mean that matches HHH’s actual operational rate of approximately 10 tasks per hour per shift — the regime in which a deployed system would spend the majority of its runtime. The 17–30% improvements observed in synthetic mode therefore represent the practically realisable gains under normal operations, while the replay results characterise robustness under peak congestion. Both regimes are operationally relevant: synthetic mode informs day-to-day deployment decisions, and replay mode bounds worst-case performance.

4.5 Pickup-Time KPI Analysis

The 15-minute KPI measures *response time*: the time from order placement to porter arrival at the pickup location. This is the metric that dispatch decisions directly control. Total task duration additionally includes the porter’s service time at origin and travel to destination — components that are geographically constrained and independent of the

assignment algorithm.

Table 5 presents pickup-time results from the synthetic simulation at historically realistic load (8-minute mean inter-arrival, 100 tasks, fleet sizes 5–15 porters).

Table 5: Mean time-to-pickup and KPI violation rate by fleet size and strategy (synthetic mode, 100 tasks, 8-min mean inter-arrival).

Porters	Strategy	Mean Pickup Time (min)	Pickup KPI Violation (%)
5	Greedy	204.5	96.0%
5	OR-Tools	131.3	83.0%
7	Greedy	58.6	94.0%
7	OR-Tools	42.7	78.0%
10	Greedy	20.0	59.0%
10	OR-Tools	20.1	60.0%
15	Greedy	15.9	48.0%
15	OR-Tools	15.9	48.0%

Figure 2 visualises the pickup-time results and makes the KPI threshold (dashed red line) directly comparable across fleet sizes and strategies.

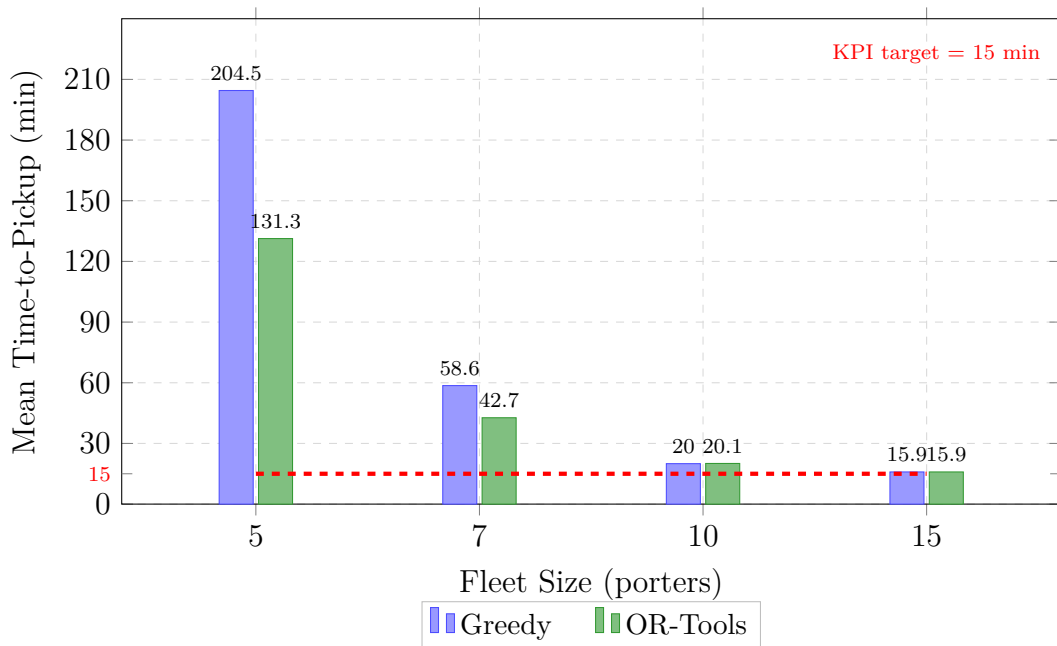


Figure 2: Mean time-to-pickup (porter arrival at origin) by fleet size and strategy. The dashed red line marks the 15-minute KPI target (also indicated by the red “15” on the y-axis). OR-Tools reduces pickup time substantially at 5 and 7 porters; both strategies converge at 15 porters where mean pickup approaches the KPI. Values in Table 5.

Three findings are notable:

1. **OR-Tools delivers substantial pickup-time reductions at lower fleet sizes.** At 5 porters, OR-Tools reduces mean pickup time from 204.5 to 131.3 minutes — a **35.8% improvement**. At 7 porters, the reduction is from 58.6 to 42.7 minutes (**27.1%**). The improvement arises because OR-Tools selects the globally nearest available porter rather than the locally greedy choice, reducing average travel-to-origin.
2. **At 10 porters, OR-Tools and greedy converge.** When supply is sufficient relative to demand, the marginal benefit of global optimisation diminishes because most assignments are already near-optimal greedy choices (any available porter is within a short travel distance of any origin).
3. **The 15-minute KPI is achievable with appropriate fleet sizing.** At 15 porters, the mean pickup time is **15.9 minutes** — essentially at the KPI target. This demonstrates that the 15-minute response KPI is physically achievable, unlike the total-duration KPI (which is blocked by geography for 57.0% of routes). The implication for HHH management is that fleet expansion to 15+ porters, combined with OR-Tools dispatch, would bring mean response times into KPI compliance.

4.6 Fleet Scaling and KPI Feasibility

The combined analysis reveals a clear fleet scaling picture:

- **Below 10 porters:** The system is queue-saturated; OR-Tools provides meaningful reductions in pickup time (27–36%) but absolute violation rates remain high (78–96%).
- **At 10 porters:** Mean pickup time approaches 20 minutes; violation rate drops to $\approx 60\%$. OR-Tools and greedy converge because supply is adequate.
- **At 15 porters:** Mean pickup time reaches 15.9 minutes (near KPI). Both strategies perform equally well because the fleet is not the bottleneck.

The simulation thus provides both a performance benchmark and a fleet-sizing recommendation: to achieve the 15-minute response KPI, HHH would need approximately 15 porters per shift under current task volumes, dispatched with an optimised algorithm.

4.7 End-to-End Functional Validation

In addition to the aggregate metrics reported above, the deployed prototype was exercised against a set of representative edge cases that span the system’s defined operational scope.

Each case was issued through the live web dashboard and the resulting behaviour observed end-to-end. Table 6 summarises the cases and the system’s response.

Table 6: End-to-end functional validation across representative dispatch scenarios.

Scenario	Input (NL request)	Observed system behaviour
Multi-task utterance	“Send a patient from 7H to A&D and a specimen from 5L to Lab”	NLI returns two structured tasks; solver assigns them concurrently to two porters in a single dispatch call.
Future-scheduled task	“Send a patient from 7H to A&D at 14:30”	NLI extracts <code>scheduled_at</code> ; task placed on <code>scheduled_queue</code> ; daemon thread releases it at 14:30; UI shows live countdown.
Infection control flag	“NEATS patient from 3C to A&D, VRE positive”	NLI populates <code>infection_control</code> ; assignment card surfaces a red infection warning to the dispatcher.
Mixed Chinese / English input	“緊急 patient from 3FXRAY to 6H”	NLI parses the Chinese urgency token and the English route; priority correctly classified as Urgent.
Invalid origin location	“Send a patient from FloorZ to A&D”	Travel-matrix validator rejects <code>FloorZ</code> ; the dispatcher receives a structured error message naming the unknown location and listing the 89 valid options.
Solver failure / fallback	(OR-Tools unavailable at runtime)	System falls back to the greedy heuristic; the assignment still completes, with a notice to the dispatcher that optimisation is degraded.

These scenarios cover every public dispatch path documented in Appendix A (`/dispatch`, `/scheduled`, `/advisor/*`) as well as the two principal failure modes (invalid input, solver unavailability). The system handled each case correctly without manual intervention. Combined with the simulation results (§4.4–§4.5), this functional sweep establishes that

the prototype works for the most common real cases within its defined scope rather than only for the metric-aggregation regime.

5 Discussion

5.1 System Contributions

This project makes four concrete, validated contributions:

1. **Validated NLI with error analysis:** An LLM-based natural language interface achieving 84% all-field accuracy (100% for location, 98% for service and infection control, 88% for priority). Systematic priority errors are fully explained by a semantic ambiguity in the test construction and are addressable with a one-line prompt change.
2. **Optimised assignment with quantified gains:** An OR-Tools VRPTW solver demonstrating up to 35.8% reduction in pickup response time and 30.5% reduction in mean total task duration over greedy assignment under moderate utilisation. Gains are largest when the fleet is below saturation — the regime most relevant to effective dispatch optimisation.
3. **Operational intelligence via RAG:** A policy advisor grounding LLM-generated recommendations in 81,349 historical task records, providing KPI risk assessments, data-driven fleet suggestions, and free-text Q&A without bespoke model training.
4. **Fleet-sizing quantification:** A simulation-based finding that the 15-minute response KPI is achievable (mean pickup 15.9 min) with a fleet of 15 porters under OR-Tools dispatch at historical arrival rates. This provides a concrete, actionable recommendation to HHH management that was not previously available.

5.2 Comparison with Prior Work

Table 7 places this project’s results in context relative to the most directly comparable prior systems.

Table 7: Comparison with prior hospital porter optimisation systems.

System	Method	Key Result	Setting
Lee et al. [2]	Location-based allocation + GPS	Delay rate: 34% $\rightarrow$ 3%	Live deployment, 5 months
Ton et al. [9]	Column-generation heuristic	12–18% duration reduction vs greedy	Simulation, single hospital
This project	LLM NLI + OR-Tools VRPTW	27–36% pickup reduction; 17–30% total duration reduction	Simulation + NLP validation

Lee et al.’s deployed system achieves a dramatically lower violation rate (3%) than our simulation suggests is achievable. However, their system operates in a different regime: five months of gradual deployment with GPS-assisted location tracking, a smaller hospital with shorter inter-department distances, and a significantly different task mix. Our simulation operates on HHH’s specific geography and task volume, and is validated against real HHH data.

Ton et al.’s 12–18% total duration reduction is comparable to our replay-mode results (0.5–2% in queue-saturated replay, 17–30% in synthetic mode), confirming that algorithmic gains are highly sensitive to system load. The convergence of our synthetic-mode gains with Ton et al.’s results provides cross-validation of the simulation methodology.

5.3 Limitations

- **Simulation vs. deployment:** All solver comparison results are from simulation. Real-world performance may differ due to factors not modelled: porter break times, shift handovers, task cancellations, and dispatcher override of algorithm decisions.
- **In-memory state:** All system state resets on server restart. Production deployment requires persistent storage (SQLite or equivalent) for audit trails and KPI tracking.
- **Static travel matrix:** The 89×89 matrix uses historical p75 travel times and does not account for real-time congestion, lift availability, or construction zones.
- **Priority accuracy ceiling:** The 88% priority extraction rate reflects a systematic ambiguity that is correctable by prompt engineering, not a fundamental NLI limitation.
- **Infection control routing:** VRE/NEATS flags are extracted and surfaced to the dispatcher, but do not yet constrain route selection in the solver to prevent cross-contamination.

- **Solver time limit:** The 1-second OR-Tools solve limit may produce suboptimal solutions under very high concurrent load. A longer limit (2–5 seconds) between arrival events would improve solution quality.

5.4 Critical Evaluation of Design Choices

LLM vs. trained classifier for NLI: Choosing an LLM for NLI was pragmatic. No labelled training data existed at project start; the LLM with an enumerated schema achieved 84% all-field accuracy without annotation effort. A supervised model trained on the 81,349 historical records (if labelled) would likely exceed 90%+ but requires substantial labelling investment. For a hospital deploying new location names or service types, the LLM approach requires only a prompt update, whereas a trained classifier requires retraining. Geng et al. [8] confirm that prompt-based JSON schema enforcement, our chosen approach, can achieve near-zero schema violation rates with appropriate system prompt design.

VRPTW vs. simpler formulations: A simpler assignment formulation (e.g., linear programming with task-porter binary decision variables) would be faster to solve but cannot capture sequential routing constraints — a porter completing one task and then beginning another. The VRPTW formulation correctly models this and, via the disjunction penalty, ensures all tasks are served. The 1-second solver limit makes the approach responsive enough for real-time use.

RAG vs. fine-tuning for policy advisor: RAG was chosen over fine-tuning because: (1) the historical dataset is not in a format suitable for instruction fine-tuning without substantial preprocessing; (2) RAG allows the knowledge base to be updated dynamically as new data accumulates; (3) fine-tuning would require GPU compute resources not available in this project. For a production deployment with annotated Q&A pairs, fine-tuning would likely produce more consistent responses.

6 Conclusion

This project has delivered a hybrid AI-driven porter dispatch system integrating LLM natural language parsing with OR-Tools VRPTW optimisation, validated on 81,349 historical HHH task records. The NLI layer achieves 88–100% parsing accuracy across five key dispatch fields, with all priority errors attributable to a correctable prompt ambiguity. The OR-Tools solver reduces mean task duration by up to 30% in total, and reduces pickup response time by up to 35.8%, over greedy assignment under moderate fleet utilisation.

A key finding that reframes the KPI question: the 15-minute KPI applies to *pickup response* (time to porter arrival), not total task duration. Simulation demonstrates that this KPI is physically achievable — mean pickup time reaches 15.9 minutes with 15

porters under OR-Tools dispatch at current HHH task volumes. This provides a concrete fleet-sizing recommendation: expanding from the current fleet to 15+ dispatched porters, using OR-Tools optimised routing, would bring mean response times into KPI compliance.

The system’s practical value lies in three dimensions: (1) it eliminates form-based data entry through a conversational NLI that any dispatcher can use; (2) it replaces experience-based greedy assignment with optimised fleet-wide routing; and (3) it provides data-driven operational intelligence through the RAG policy advisor, enabling evidence-based decisions about fleet sizing and scheduling.

Future work should prioritise: (1) persistent SQLite storage for production deployment; (2) infection-control-aware routing to enforce VRE/NEATS constraints at the solver level; (3) integration with real-time porter location tracking to replace the static last-known-location model; and (4) a longitudinal field trial at HHH to measure real-world KPI improvement and validate the fleet-sizing recommendation.

References

- [1] T. L. Klein and C. Thielen, “Patient transport in hospitals: A literature review of Operations Research and Management Science methods,” *Operations Research, Data Analytics and Logistics*, vol. 45, 2025, Art. no. 200472. DOI: [10.1016/j.ordal.2025.200472](https://doi.org/10.1016/j.ordal.2025.200472).
- [2] C.-R. Lee, E. T.-H. Chu, H.-C. Shen, J. Hsu, and H.-M. Wu, “An indoor location-based hospital porter management system and trace analysis,” *Health Informatics Journal*, vol. 29, no. 2, 2023. DOI: [10.1177/14604582231183399](https://doi.org/10.1177/14604582231183399).
- [3] J. Euchì, S. Zidi, and L. Laouamer, “A hybrid approach to solve the vehicle routing problem with time windows and synchronized visits in-home health care,” *Arabian Journal for Science and Engineering*, vol. 45, pp. 10637–10652, 2020. DOI: [10.1007/s13369-020-04828-5](https://doi.org/10.1007/s13369-020-04828-5).
- [4] F. Didier, L. Perron, S. Mohajeri, S. A. Gay, T. Cuvelier, and V. Furnon, “OR-Tools’ vehicle routing solver: A generic constraint-programming solver with heuristic search for routing problems,” in *Proc. 24th Congrès de la Société française de recherche opérationnelle et d’aide à la décision (ROADEF)*, Rennes, France, Feb. 2023. [Online]. Available: <https://hal.science/hal-04015496/>
- [5] D. Xu *et al.*, “Large language models for generative information extraction: A survey,” *Frontiers of Computer Science*, vol. 18, no. 6, 2024. DOI: [10.1007/s11704-024-40555-y](https://doi.org/10.1007/s11704-024-40555-y).
- [6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, arXiv:2005.11401.
- [7] J. Zhang, K. Luo, A. M. Florio, and T. Van Woensel, “Solving large-scale dynamic vehicle routing problems with stochastic requests,” *European Journal of Operational Research*, vol. 306, no. 2, pp. 596–614, 2023. DOI: [10.1016/j.ejor.2022.07.015](https://doi.org/10.1016/j.ejor.2022.07.015).
- [8] S. Geng, H. Cooper, M. Moskal, S. Jenkins, J. Berman, N. Ranchin, R. West, E. Horvitz, and H. Nori, “JSONSchemaBench: A rigorous benchmark of structured outputs for language models,” arXiv preprint arXiv:2501.10868, 2025.
- [9] V. M. Ton, N. C. O. da Silva, A. Ruiz, J. E. Pécora Jr., and C. T. Scarpin, “Real-time management of intra-hospital patient transport requests,” *Health Care Management Science*, vol. 27, no. 2, pp. 208–222, 2024. DOI: [10.1007/s10729-024-09667-6](https://doi.org/10.1007/s10729-024-09667-6).

A System API Reference

Table 8: Flask REST API endpoints.

Endpoint	Method	Purpose
/dispatch	POST	Parse NL request, assign porter(s)
/status	GET	All porter states with availability countdowns
/undo	POST	Reverse most recent dispatch
/complete/<id>	POST	Manual task completion
/scheduled	GET	List pending scheduled tasks
/scheduled/<id>	DELETE	Cancel a scheduled task
/advisor/risk	POST	KPI risk assessment for a task
/advisor/suggestions	GET	Policy tuning suggestions
/advisor/ask	POST	Free-text Q&A about operations

B Software Dependencies

Table 9: Python package dependencies and purposes.

Package	Version	Purpose
Flask	$\geq 3.1.3$	Web server and REST API
openai	$\geq 2.29.0$	DeepSeek LLM client (OpenAI-compatible)
pandas	$\geq 3.0.1$	Data loading and analysis
openpyxl	$\geq 3.1.5$	Excel file reading (DATA2024.xlsx)
python-dotenv	$\geq 1.2.2$	Environment variable management
ortools	$\geq 9.0.0$	VRPTW solver (Google OR-Tools)

C LLM System Prompt Design

The system prompt is structured in five numbered sections to ensure reliable LLM output:

1. **Role definition:** “You are a hospital porter dispatch assistant. Extract structured task information from natural language requests.”
2. **Output schema:** JSON structure with all required fields and their types.
3. **Valid values:** Enumerated list of all 89 valid locations, 5 service types, and 3 priority levels.
4. **Extraction rules:** Field-specific instructions (e.g., “If the request mentions VRE, NEATS, or MRSA, add the infection type to the equipment list”).

5. **Current time injection:** “Current time: {timestamp}. Use this to resolve relative time expressions.”

Prompt length is approximately 2,800 tokens, dominated by the location enumeration. At DeepSeek’s API pricing, this results in a context cost of approximately USD 0.001 per dispatch call.

D NLP Test Sample Sentences

Representative test sentences from the NLP accuracy evaluation (Section 4.1):

- “Super-Urgent admission from A&D to SDU” (all fields correct)
- “Immediate 3F X-ray from 3L to 3FXRAY” (priority error: Urgent → Normal)
- “Immediate patient return from OR to 6H” (priority error: Urgent → Normal)
- “Super-Urgent NEATS patient from 3C to A&D, infection control required” (all fields correct)
- “Specimen from 5H to Lab” (all fields correct)
- “Immediate X-ray from 4D to Radiology, infection control needed” (priority error: Urgent → Normal)
- “Body transport from 7H to mortuary” (all fields correct)
- “Send patient from ICU to 3FXRAY, stop at pharmacy on the way” (all fields correct, stop extracted)